

MY AWS PROJECT

Terraform AWS VPC Peering + ALB

Connecting two AWS networks privately and load-balancing traffic across three servers — built with Terraform

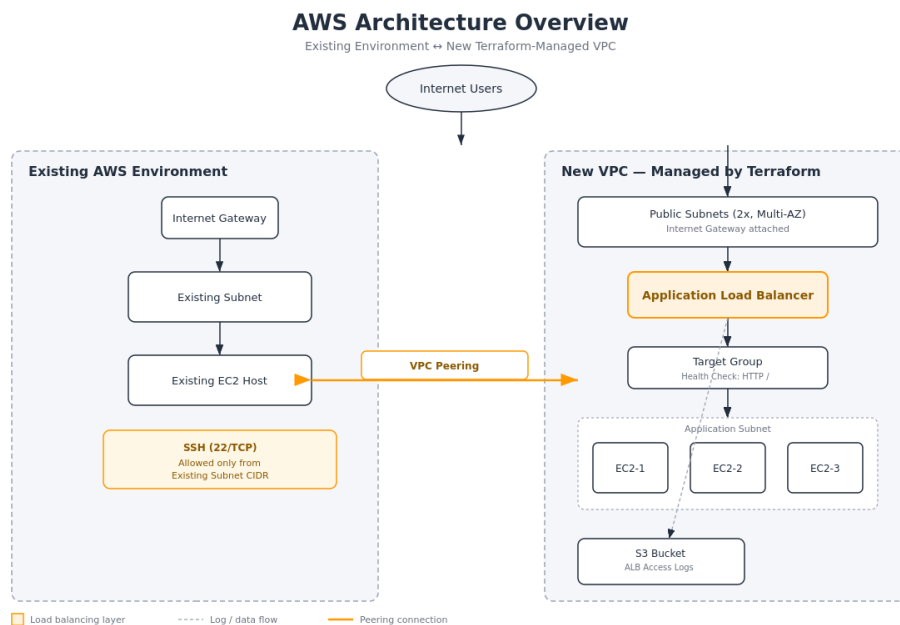
Aritra Mondal · Linux Administrator, moving into Cloud Engineering · [View on GitHub](#)

What This Project Does

Imagine a company already has some servers running on AWS. Now they want to add new servers — but keep them in a separate, more secure network, while still letting the two networks talk to each other privately.

That's exactly what this project builds. I created a brand-new, isolated network on AWS, connected it privately to the company's existing network, and put three servers behind a load balancer so traffic is spread evenly and the app stays available even if one server has a problem. Everything was built using Terraform, a tool that lets you describe your entire cloud setup in code instead of clicking through the AWS website by hand.

How It Works



The existing AWS network (left) and the new network I built (right), joined by a private connection.

- **VPC Peering** is a private tunnel between the two networks — traffic flows between them without ever touching the public internet.
- **The Load Balancer** sits at the front door and spreads incoming traffic evenly across all three servers, so no single server gets overloaded.
- The load balancer keeps checking each server's health, so if one ever goes down, traffic automatically stops going to it.
- All requests handled by the load balancer are logged and saved to cloud storage (S3), so there's always a record to look back on.
- **Security:** Only the existing, trusted network is allowed to log in to the new servers — the public internet cannot.

Key Terms, in Plain Language

Term	What it means
VPC	A private, isolated network inside AWS — like your own fenced-off section of the cloud.
Subnet	A smaller section within a VPC where specific resources (like servers) live.
VPC Peering	A private link that connects two separate VPCs so they can talk to each other.
Security Group	A firewall for your servers — it decides what traffic is allowed in or out.
Load Balancer (ALB)	A traffic director that spreads requests across multiple servers.
Health Check	A regular check the load balancer does to confirm a server is working.
S3 Bucket	AWS's cloud storage service — used here to save the load balancer's logs.

How It Was Built

Rather than creating each piece manually in the AWS console, I wrote the entire setup as Terraform code. This means the whole network, servers, and load balancer can be recreated automatically, consistently, and in minutes — a core skill in modern cloud infrastructure work.

Terraform Deployment Flow

Order in which resources are provisioned



The order in which Terraform builds each part of the infrastructure.

Repository Structure

```
[root@main-ec2 project1-tf]# tree
```

```
.
├── ec2_instance.tf
├── import.tf
├── internet_gateway.tf
├── load_balancer.tf
├── network.tf
├── output.tf
├── route_table.tf
├── s3_bucket.tf
├── security_group.tf
├── terraform.tf
├── terraform.tfstate
├── terraform.tfstate.backup
├── variables.tf
└── vpc_peer.tf
```

```
1 directory, 14 files
```

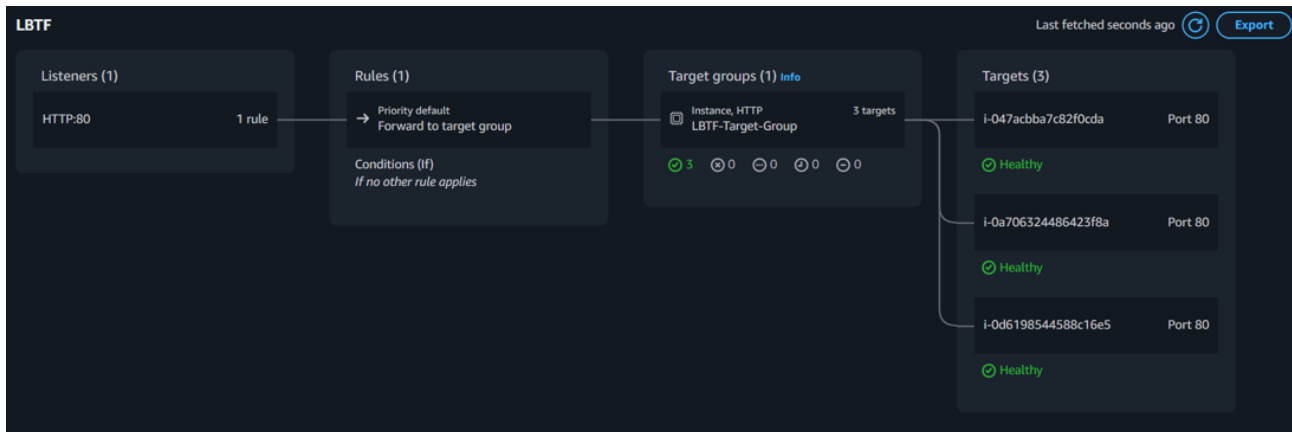
```
[root@main-ec2 project1-tf]# |
```

Screenshots

```
Apply complete! Resources: 27 added, 0 changed, 0 destroyed.
```

Outputs:

```
instances = <<EOT
Instances Deployed -
node1 : 10.2.0.229 : ec2-admin
node2 : 10.2.0.79 : ec2-admin
node3 : 10.2.0.245 : ec2-admin
URL : LBTF-984376798.ap-south-1.elb.amazonaws.com
EOT
[root@main ec2-project1 ~]# cat inventory.py
```



← ↻ 🏠 ⚠ Not secure lbtf-984376798.ap-south-1.elb.amazonaws.com

Hellow from ip-10-2-0-79.ap-south-1.compute.internal on 10.2.0.79...

← ↻ 🏠 ⚠ Not secure lbtf-984376798.ap-south-1.elb.amazonaws.com

Hellow from ip-10-2-0-229.ap-south-1.compute.internal on 10.2.0.229...

← ↻ 🏠 ⚠ Not secure lbtf-984376798.ap-south-1.elb.amazonaws.com

Hellow from ip-10-2-0-245.ap-south-1.compute.internal on 10.2.0.245...

What I Learned

- How to write Infrastructure as Code with Terraform instead of manual, click-based setup.
- How two AWS networks can be securely connected using VPC Peering.
- How to design firewall rules that follow the principle of least privilege.
- How a load balancer improves both availability and reliability for an application.
- How to launch multiple identical servers efficiently from a single piece of code.

What's Next

- Add HTTPS so traffic to the load balancer is encrypted.
- Add basic monitoring and alerts to catch issues early.
- Automatically add or remove servers based on traffic load.
- Deploy a real sample application on the servers to see the whole setup in action.

Full code, diagrams, and documentation: github.com/aritramondal-1711/terraform-aws-ec2-alb-project